Chosen Improved Greedy Algorithm

Fast Cryptography Laboratory

Master Thesis Draft

Submitted by:
Hong-Sheng Huang

Advisor: Dr. Tung Chou, Associate Research Fellow

March, 17, 2026

Glossary

mat Matrix

inv Inverse

minm Minimum Cost

per Permutation Matrix

seq Sequence List

Contents

1 Introduction

1.1 Motivation

Quantum cryptography, as the name suggests, utilizes the cryptographic mechanisms underlying quantum circuits. To improve efficiency, it is essential to find more optimized implementations for these circuits. In quantum computing, many applications rely heavily on linear reversible circuits; therefore, deploying efficient methods to reduce their computational cost is key to providing more practical algorithms.

Currently, most symmetric crypto-systems are constructed using a significant number of CNOT gates in quantum circuit. Drawing inspiration from the field of Very Large Scale Integration (VLSI), we aim to reduce quantum algorithm costs by focusing on circuit depth as our primary metric, targeting linear scaling.

In this work, we propose an enhancement to the randomized greedy algorithm using heuristic methods to achieve the shallowest possible depth with the minimum number of CNOT gates. This approach not only guarantees a speedup in the execution of quantum cryptographic algorithms but also reduces the impact of quantum noise and hardware constraints on the computation results.

1.2 Problem Statement

Our research addresses the challenge of finding the optimal (minimum) depth for constructing the matrix components of symmetric ciphers in a quantum computing environment. All matrices are in $\mathbb{F}_2$.

2 Related Work

3 Preliminaries

3.0.1 Notations

- M: Given an $n \times n$ input matrix A, our greedy algorithm seeks to identify a sequence of operations that transforms A into a permutation matrix.
- d: The circuit depth achieved when matrix A is transformed into a permutation matrix.
- M': The matrix cost function is derived from the inverse of matrix A.
- SIZE: Since we are specifically considering square block matrices, the matrix size is determined by retrieving the row or column length using the built-in 'length()' function.
- L_r, L_c: Row and column operation sequences are stored in separate layer lists for each iteration.
- Ls_r, Ls_c: We maintain row-layer and column-layer lists to store the specific operations assigned to each discrete circuit layer.
- L_{row}, L_{col}: A list of remaining operations is updated after each step to exclude those already applied to the matrix.
- $L_{row}^{cost}, L_{col}^{cost}$: For every operation in L_{row} and L_{col}, the algorithm calculates and stores the resulting matrix cost function in a lookup list.
- row_{op}, col_{op}: The collection of applied row and column operations is stored to construct the final sequence list and to verify the correctness of the transformation.
- p: A control parameter is used to select the matrix cost function type. Supported functions include linear sum (1), square (2), cubic (3), quartic (4), and logarithmic (- 1), which can be selected via their respective numerical identifiers.
- over_depth: An acceptance flag is utilized to evaluate the current circuit depth; it returns 'True' if the depth d exceeds the current minimum depth, and 'False' otherwise.

- close_permu: A termination flag monitors whether the matrix is nearing a permutation state. If the current matrix can be transformed into a permutation matrix within a single layer, the flag is set to True; otherwise, it remains False.
- minm: The algorithm identifies the current minimum cost across all candidates and selects the operation corresponding to this optimal value.
- origin, inverse: At the start of the process, a copy of the input matrix is retained for final verification. Simultaneously, the matrix inverse is calculated to facilitate the evaluation of the cost function.
- $visi_{row}, visi_{col}$: A visited list tracks the selected row and column indices; once a position is chosen, it is excluded from future iterations to prevent redundant operations.
- select_list: The selection list stores the set of candidate operations available for the current optimization step.
- H_r, H_c: The matrix cost function is evaluated from both a row perspective and a column perspective to determine the most efficient operation.
- B_{row}, B_{col}: The algorithm maintains a collection of row and column operations for both the Row/Columnwise and Parallel greedy approaches; it identifies the available operators and selects the one with the lowest computational cost.
- $best_{row}^{cost}, best_{col}^{cost}$: The algorithm maintains a collection of cost values for each potential operation in B_{row} and B_{col} to facilitate the selection of the optimal operator.
- stuck_counter: A counter is used to record the number of consecutive stalls (stuck iterations) to detect when the algorithm has reached a local optimum.
- max_stuck_iteration: A stagnation threshold is defined to limit the number of consecutive stalls; by default, this value is set to 3.
- escapeCandidates: The algorithm also maintains a collection of non-optimal operators, representing candidates whose cost values exceeded the current minimum.

- best_escape: To resolve local minima, the algorithm identifies the optimal candidate from the escapeCandidates list—after sorting it in ascending order—and adds it to the select_list to resume the search.
- occur_time: The acceptance counter records the total number of iterations in which an operation was successfully selected and applied.
- minm_size: A threshold condition is implemented to ensure that the circuit depth (total number of layers) does not exceed the predefined limit.
- reduce: A copy of the initial matrix is retained to verify that the sequence of row and column operations (row_{op} and col_{op}) successfully transforms the origin matrix into the target representation.
- size_minm: The algorithm records the final circuit depth to document the efficiency of the optimized sequence.
- per: The algorithm records the final matrix state to generate a permutation list, which defines the final mapping of the quantum registers.
- seq: The algorithm maintains a comprehensive log of all applied row and column operations (row_{op} and col_{op}), which constitutes the final quantum circuit sequence.
- layers: The algorithm maintains a record of the sets Ls_c and Ls_r, which contain the column and row operations assigned to each parallel layer.
- correct: A validation flag is employed to confirm that the combined operation sequence and layer assignments successfully transform the original matrix into a permutation matrix.
- greedy: An algorithmic selection parameter allows the user to specify which greedy strategy (e.g., Row, Column, Row_or_Col or Parallel) should be employed during the synthesis process.

4 Greedy Algorithms for CNOT Gate Composition

In our work, we adopt the heuristic method to find the most shallow depth for CNOT gate, after we survey on
Building on the work of **?**, who proposed an improved depth-oriented algorithm for synthesizing ancilla-free, low-depth CNOT circuits, we developed four distinct greedy strategies to further explore depth-optimized results.
By utilizing different matrix representations and search heuristics, we implemented the Row, Column, Row-or-Column, and Parallel algorithms.
The pseudocode and implementation logic for each approach are detailed in the following sections.

4.1 Row/Col

This strategy aims to drive the matrix toward the 'can_one' condition. Once the 'one' flag is triggered, the algorithm shifts its priority exclusively to row operations to facilitate the final reduction into a permutation matrix.

4.2 Row-or-Col

This strategy evaluates both row and column operations and selects the layer that yields the minimum matrix cost. In cases where the costs are identical, the algorithm defaults to the column-wise operation.
If only one set of operations is available, the algorithm selects the existing set.
To address the inherent risk of local minima in greedy heuristics, we implement a stuck threshold. If the cost fails to decrease for three consecutive iterations, the algorithm accepts a sub-optimal operation that increases the cost, thereby enabling an escape from the local minimum.

4.3 Parallel

The Parallel Greedy strategy aggregates both row and column operations into a unified 'select_list'.
In cases where one operation type is unavailable, the algorithm proceeds with the existing candidates. Similar to the Row-or-Column approach, a stagnation

Algorithm 1 Row/Col Greedy

Input: $M \in \mathbb{F}_2^{n\times n}$, $M' \in \mathbb{F}_2^{n\times n}$, L_r, L_c, Ls_r, Ls_c, p and over_depth
Output: $Ls_r, Ls_c, row_{op}, col_{op}$ that implement permutation matrix with length d

SIZE ← length(mat)
minm ← system.float_info.max
select_list ← list()
$visi_{row}$ ← [0], $visi_{col}$ ← [0]
close_permu ← False
LIMIT ← LATEST_MINM_DEPTH
while *mat* != Permutation **do**
    select_list.clear()
    L_{row}.clear(), L_{col}.clear(), L_{row}^{cost}.clear(), L_{col}^{cost}.clear()
    H_r, H_c, minm ← functionCost(M, M', p)
    if !close_permu **then**
        $L_{col} \leftarrow L\_Collection(visi_{row})$
        select_list, minm ← $Row\_Op\_Sel(L_{col}, L_{col}^{cost}, M, M', p)$
    $L_{row} \leftarrow L\_Collection(visi_{row})$
    function AVAILABLEROWCOLLECTION($visi_{row}$, SIZE)
        return L_{row}
    function AVAILABLEROWOPERATORSELECTION(L_{row}, L_{row}^{cost}, *mat*, *inv*, *p*)
        return select_list, minm
    function AVAILABLEOPERATOREXECUTION(select_list, L_r, L_c, Ls_r, Ls_c, *mat*, *inv*, row_{op}, col_{op}, $visi_{row}, visi_{col}$, *d*, SIZE, one)
        return L_r, L_c, Ls_r, Ls_c, *mat*, *inv*, row_{op}, col_{op}, $visi_{row}, visi_{col}$, *d*, one
    if $d >$ LIMIT **then**
        $flag \leftarrow$ True
        return
return $L_r, L_c, Ls_r, Ls_c, mat, row_{op}, col_{op}, d$, flag

Algorithm 2 Row_or_Col Greedy

Input: $mat_{n\times n}$, $inv_{n\times n}$, L_r, L_c, Ls_r, Ls_c, p and $flag$
Output: $Ls_r, Ls_c, row_{op}, col_{op}$ that implement permutation matrix with length d

SIZE ← length(mat)
minm, $best^{cost}_{row}, best^{cost}_{col}$ ← system.float_info.max
select_list ← list()
$visi_{row}$ ← [0], $visi_{col}$ ← [0]
LIMIT ← LATEST_MINM_DEPTH
stuck_counter ← 0, max_stuck_iterations ← 3
while *mat* != Permutation **do**
    select_list.clear()
    L_{row}.clear(), L_{col}.clear(), L^{cost}_{row}.clear(), L^{cost}_{col}.clear(), B_{row}.clear(), B_{col}.clear()
    if p != 1 **then**
        H_r ← costMat(*mat*, *p*) + costMat(inv^T, *p*)
        H_c ← costMat(mat^T, *p*) + costMat(*inv*, *p*)
        minm ← max(H_r, H_c)
    else
        minm ← costMat(*mat*, *p*) + costMat(*inv*, *p*)
    function AVAILABLEROWCOLLECTION($visi_{row}$, SIZE)
        return L_{row}
    function MODIFIEDAVAILABLEROWOPERATORSELECTION(L_{row}, L^{cost}_{row}, *mat*, *inv*, *p*, minm)
        return $B_{row}, best^{cost}_{row}$
    function AVAILABLECOLCOLLECTION($visi_{col}$, SIZE)
        return L_{col}
    function MODIFIEDAVAILABLECOLOPERATORSELECTION(L_{col}, L^{cost}_{col}, *mat*, *inv*, *p*, minm)
        return $B_{col}, best^{cost}_{col}$
    is_stuck = length(B_{row}) == 0 ∧ length(B_{col}) == 0
    if is_stuck **then**
        stuck_counter += 1
    else
        stuck_counter ← 0
    if (stuck_counter ≥ max_stuck_iterations) OR (is_stuck ∧ $\sum visi_{row}$ == 0 ∧ $\sum visi_{col}$ == 0) **then**
        escapeCandidates.clear()
        function AVOIDLOCALMINIMAAVAILABLEOPERATORSELECTION($L_{row}, L_{col}, mat, inv, p$, escapeCandidates)
            return select_list, minm
    if $best^{cost}_{row} < best^{cost}_{col}$ **then**
        select_list ← B_{row}
        minm ← $best^{cost}_{row}$
    else if $best^{cost}_{row} > best^{cost}_{col}$ **then**
        select_list ← B_{col}
        minm ← $best^{cost}_{col}$
    else
        select_list ← B_{col} if len(B_{col}) > 0 else B_{row}
        minm ← $best^{cost}_{col}$ if len(select_list) > 0 else minm
    function AVAILABLEOPERATOREXECUTION(select_list, L_r, L_c, Ls_r, Ls_c, *mat*, *inv*, row_{op}, col_{op}, $visi_{row}, visi_{col}$, *d*, SIZE)
        return L_r, L_c, Ls_r, Ls_c, *mat*, *inv*, row_{op}, col_{op}, $visi_{row}, visi_{col}$, *d*
    if *d* > LIMIT **then**
        $flag$ = True
        return
return $L_r, L_c, Ls_r, Ls_c, mat, row_{op}, col_{op}, d$, flag

threshold is utilized to escape local minima; if no progress is made for three iterations, the algorithm accepts cost-increasing moves.
The primary distinction of the Parallel strategy lies in its late-stage optimization: when the matrix approaches a permutation state, the 'one' flag is activated, restricting the search to column-wise operations to finalize the circuit.

Algorithm 3 Parallel Greedy

Input: $mat_{n\times n}$, $inv_{n\times n}$, L_r, L_c, Ls_r, Ls_c, p and $flag$
Output: $Ls_r, Ls_c, row_{op}, col_{op}$ that implement permutation matrix with length d

SIZE ← length(mat)
minm, $best^{cost}_{row}, best^{cost}_{col}$ ← system.float_info.max
select_list ← list()
$visi_{row}$ ← [0], $visi_{col}$ ← [0]
LIMIT ← LATEST_MINM_DEPTH
one ← False
stuck_counter ← 0, max_stuck_iterations ← 3
while mat != Permutation **do**
    select_list.clear()
    L_{row}.clear(), L_{col}.clear(), L^{cost}_{row}.clear(), L^{cost}_{col}.clear(), B_{row}.clear(), B_{col}.clear()
    if p != 1 **then**
        H_r = costMat(mat, p) + costMat(inv^T, p)
        H_c = costMat(mat^T, p) + costMat(inv, p)
        minm = max(H_r, H_c)
    else
        minm = costMat(mat, p) + costMat(inv, p)
    if !one **then**
        function AVAILABLEROWCOLLECTION($visi_{row}$, SIZE)
            return L_{row}
        function MODIFIEDAVAILABLEROWOPERATORSELECTION(L_{row}, L^{cost}_{row}, mat, inv, p, minm)
            return $B_{row}, best^{cost}_{row}$
    function AVAILABLECOLCOLLECTION($visi_{col}$, SIZE)
        return L_{col}
    function MODIFIEDAVAILABLECOLOPERATORSELECTION(L_{col}, L^{cost}_{col}, mat, inv, p, minm)
        return $B_{col}, best^{cost}_{col}$
    is_stuck = length(B_{row}) == 0 $\wedge$ length(B_{col}) == 0
    if is_stuck **then**
        stuck_counter += 1
    else
        stuck_counter ← 0
    if (stuck_counter $\geq$ max_stuck_iterations) OR (is_stuck $\wedge \sum visi_{row} == 0 \wedge \sum visi_{col}$ == 0) **then**
        escapeCandidates.clear()
        function AVOIDLOCALMINIMAAVAILABLEOPERATORSELECTION(L_{row}, L_{col}, mat, inv, p, escapeCandidates)
            return select_list, minm
    else
        select_list ← $B_{row} + B_{col}$
        if $best^{cost}_{row} < best^{cost}_{col}$ **then**
            minm ← $best^{cost}_{row}$
        else if $best^{cost}_{row} > best^{cost}_{col}$ **then**
            minm ← $best^{cost}_{col}$
        else
            minm ← $best^{cost}_{col}$ if len(select_list) > 0 else minm
    function AVAILABLEOPERATOREXECUTION(select_list, L_r, L_c, Ls_r, Ls_c, mat, inv, row_{op}, col_{op}, $visi_{row}, visi_{col}$, d, SIZE, one)
        return L_r, L_c, Ls_r, Ls_c, mat, inv, row_{op}, col_{op}, $visi_{row}, visi_{col}$, d, one
    if $d >$ LIMIT **then**
        $flag$ ← True
        return
return $L_r, L_c, Ls_r, Ls_c, mat, row_{op}, col_{op}, d$, flag

5 Cost Functions

The ‘costMat‘ function serves as our primary heuristic, allowing for the selection of five distinct cost metrics. These metrics—logarithmic, linear sum, square, cubic, and quartic—are controlled by a parameter p, which determines the mathematical weight assigned to the matrix reduction process.

5.1 Log

This cost function is derived from the greedy framework proposed by **?**, which utilizes the logarithm of the Hamming weight for each row as follows:

$$h_{log}(A) = \sum_i log_2(\sum_j a_{i,j})$$

5.2 Sum

In addition to the logarithmic metric, the greedy framework by **?** considers the cumulative Hamming weight of each row as follows:

$$h_{sum}(A) = \sum_i \sum_j a_{i,j}$$

5.3 Square

Following the methodology of **?**, this cost function utilizes the square of each row's Hamming weight, defined as follows:

$$h_{sq}(A) = \sum_i (\sum_j a_{i,j})^2$$

5.4 The Role of Matrix Cost Functions in Greedy Optimization

The ‘costMat‘ function is integrated into two distinct phases of our greedy algorithm: the initial candidate selection and the subsequent layer evaluation.

5.4.1 Globally

First, the algorithm evaluates the global maximum cost to establish a relaxation range; this allows the search to include a broader set of available operators rather than being restricted to the absolute minimum.

Algorithm 4 Greedy

```
⋮
while mat != Permutation do
    ⋮
    if p != 1 then
        H_r ← costMat(mat, p) + costMat(inv^T, p)
        H_c ← costMat(mat^T, p) + costMat(inv, p)
        minm ← max{H_r, H_c}
    else
        minm ← costMat(mat, p) + costMat(inv, p)
    ⋮
```

Following the greedy framework established by Shi and Feng, we define the cost metrics H_r and H_c to treat row and column operations asymmetrically. Specifically, the row Hamming weight is evaluated when performing row operations (row_{op}), while the column Hamming weight is prioritized during column operations (col_{op}).
To explore the set of linear reversible operators for synthesizing quantum cryptographic circuits, we employ the following guidance:

$$H_r = costMat(A, p) + costMat((A^{-1})^T, p)$$

$$H_c = costMat(A^T, p) + costMat(A^{-1}, p)$$

5.4.2 Locally

In the second phase, the algorithm performs a local evaluation to identify individual operators that minimize the cost function. If a candidate operator's cost is lower than the current minimum, it is accepted as a valid transformation.
Once all potential control and target qubit pairs have been visited, the resulting set of parallel operations is appended to the 'select_list' as a single circuit layer.
The availableRowOperatorSelection and availableColOperatorSelection functions influence the cost matrix calculation in the Row and Col greedy algorithms.
Similarly, the modified and avoidLocalMinima versions of these functions influence the cost matrix in the Row-or-Col and Parallel greedy algorithms.

Algorithm 5 availableRowOperatorSelection

Input: $L_{row}, L_{row}^{cost}, mat_{n\times n}$, $inv_{n\times n}, p, minm$ and select_list
Output: select_list, $minm$

for $row_{op} \in L_{row}$ **do**
    temp_mat ← row_i2j(mat, row_{op}[0], row_{op}[1])
    temp_inv ← col_i2j(inv, row_{op}[1], row_{op}[0])
    if p != 1 **then**
        L_{row}^{cost}.append(costMat(temp_mat, p)+costMat(temp_inv^T, p))
    else
        L_{row}^{cost}.append(costMat(temp_mat, p)+costMat(temp_inv, p))
for index, row_{op}^{cost} in L_{row}^{cost} **do**
    if $row_{op}^{cost} < minm$ **then**
        select_list.clear()
        select_list.append((L_{row}[index][0], L_{row}[index][1], 0))
        $minm = row_{op}^{cost}$
    else if $row_{op}^{cost} == minm$ **then**
        select_list.append((L_{row}[index][0], L_{row}[index][1], 0))
return select_list, $minm$

Algorithm 6 availableColOperatorSelection

Input: $L_{col}, L_{col}^{cost}, mat_{n\times n}$, $inv_{n\times n}, p, minm$ and select_list
Output: select_list, $minm$

for $col_{op} \in L_{col}$ **do**
    temp_mat ← col_i2j(mat, col_{op}[0], col_{op}[1])
    temp_inv ← row_i2j(inv, col_{op}[1], col_{op}[0])
    if p != 1 **then**
        L_{col}^{cost}.append(costMat(temp_mat^T, p)+costMat(temp_inv, p))
    else
        L_{col}^{cost}.append(costMat(temp_mat, p)+costMat(temp_inv, p))
for index, col_{op}^{cost} in L_{col}^{cost} **do**
    if $col_{op}^{cost} < minm$ **then**
        select_list.clear()
        select_list.append((L_{col}[index][0], L_{col}[index][1], 1))
        $minm = col_{op}^{cost}$
    else if $col_{op}^{cost} == minm$ **then**
        select_list.append((L_{col}[index][0], L_{col}[index][1], 1))
return select_list, $minm$

For the Row-or-Column and Parallel strategies, which evaluate two directional operators simultaneously, the algorithm selects the layer that minimizes the total matrix cost.

To facilitate this, we implement an intermediate storage list for candidate operations and utilize their respective cost impacts as the primary selection criterion. This represents a departure from standard operator selection, as detailed in the following modifications:

Algorithm 7 modifiedAvailableRowOperatorSelection

Input: $L_{row}, L_{row}^{cost}, mat_{n\times n}$, $inv_{n\times n}, p, minm, best_{row}^{cost}$ and B_{row}
Output: $B_{row}, best_{row}^{cost}, minm$

for $row_{op} \in L_{row}$ **do**
    temp_mat ← row_i2j(mat, row_{op}[0], row_{op}[1])
    temp_inv ← col_i2j(inv, row_{op}[1], row_{op}[0])
    if p != 1 **then**
        L_{row}^{cost}.append(costMat(temp_mat, p)+costMat(temp_inv^T, p))
    else
        L_{row}^{cost}.append(costMat(temp_mat, p)+costMat(temp_inv, p))
for index, row_{op}^{cost} in L_{row}^{cost} **do**
    if $row_{op}^{cost} < minm$ **then**
        if $row_{op}^{cost} < best_{row}^{cost}$ **then**
            B_{row}.clear()
            $best_{row}^{cost} = row_{op}^{cost}$
            if $row_{op}^{cost} == best_{row}^{cost}$ **then**
                B_{row}.append((L_{row}[index][0], L_{row}[index][1], 0))
return $B_{row}, best_{row}^{cost}$

To address the issue of local minima, we implemented a stagnation recovery mechanism. When the algorithm becomes 'stuck'—meaning no cost-reducing operations are found—the greedy constraint is temporarily relaxed.

During these instances, the algorithm accepts operations that slightly increase the current minimum cost.

This heuristic perturbation enables the search to bypass stagnant states and discover subsequent operations that lead to a more optimal global reduction.

Algorithm 8 modifiedAvailableColOperatorSelection

Input: $L_{col}, L_{col}^{cost}, mat_{n\times n}$, $inv_{n\times n}, p, minm, best_{col}^{cost}$ and B_{col}
Output: $B_{col}, best_{col}^{cost}, minm$

for $col_{op} \in L_{col}$ **do**
    temp_mat ← col_i2j(mat, col_{op}[0], col_{op}[1])
    temp_inv ← row_i2j(inv, col_{op}[1], col_{op}[0])
    if p != 1 **then**
        L_{col}^{cost}.append(costMat(temp_mat^T, p)+costMat(temp_inv, p))
    else
        L_{col}^{cost}.append(costMat(temp_mat, p)+costMat(temp_inv, p))
for index, col_{op}^{cost} in L_{col}^{cost} **do**
    if $col_{op}^{cost} < minm$ **then**
        if $col_{op}^{cost} < best_{col}^{cost}$ **then**
            B_{col}.clear()
            $best_{col}^{cost} = col_{op}^{cost}$
            if $col_{op}^{cost} == best_{col}^{cost}$ **then**
                B_{col}.append((L_{col}[index][0], L_{col}[index][1], 1))
return $B_{col}, best_{col}^{cost}$

Algorithm 9 avoidLocalMinimaRowOperatorSelection

Input: $L_{row}, mat_{n\times n}$, $inv_{n\times n}, p$ and escapeCandidates
Output: escapeCandidates

for $row_{op} \in L_{row}$ **do**
    temp_mat ← row_i2j(mat, row_{op}[0], row_{op}[1])
    temp_inv ← col_i2j(inv, row_{op}[1], row_{op}[0])
    if p != 1 **then**
        temp_cost ← costMat(temp_mat, p)+costMat(temp_inv^T, p)
    else
        temp_cost ← costMat(temp_mat, p)+costMat(temp_inv, p)
    escapeCandidates.append((temp_cost, row_{op}[0], row_{op}[1], 0))
return escapeCandidates

Algorithm 10 avoidLocalMinimaColOperatorSelection

Input: $L_{col}, mat_{n\times n}$, $inv_{n\times n}, p$ and escapeCandidates
Output: escapeCandidates

for $col_{op} \in L_{col}$ **do**
    temp_mat ← col_i2j(mat, col_{op}[0], col_{op}[1])
    temp_inv ← row_i2j(inv, col_{op}[1], col_{op}[0])
    if p != 1 **then**
        temp_cost ← costMat(temp_mat^T, p)+costMat(temp_inv, p)
    else
        temp_cost ← costMat(temp_mat, p)+costMat(temp_inv, p)
    escapeCandidates.append((temp_cost, col_{op}[0], col_{op}[1], 1))
return escapeCandidates

6 Chosen Improved Greedy Algorithm

The primary algorithm integrating these cost metrics and greedy heuristics is detailed below:

Algorithm 11 Chosen Improved Greedy Algorithm

Input: mat, matName, greedy, p, occur_time
Output: d, $Layers$

minm, minm_size ← system.float_info.max
SIZE ← len(mat)
$origin$.copy(mat)
inv ← inverse(mat)
$L_r, L_c, Ls_r, Ls_c, row_{op}, col_{op}$ ← list()
$flag \leftarrow False$
function ModifiedRandomizedGreedy(greedy, p)
    return $L_r, L_c, Ls_r, Ls_c, mat, row_{op}, col_{op}, depth, flag$
if $flag$ **then**
    return
function LastCheck(L_r, L_c, Ls_r, Ls_c)
    return Ls_r, Ls_c
$reduce$.copy($origin$)
size ← 0
function GenerateReduceMatrix($reduce$, row_{op}, col_{op}, size)
    return reduce, size
$ok \leftarrow True$
if $reduce$!= mat **then**
    $ok = False$
if $(d > minm)$ OR $(d == minm \wedge size \geq minm\_size)$ OR !ok **then**
    return
$minm \leftarrow d$
size_minm ← size
per ← list()
function GeneratePermutationList(SIZE, mat, per)
    return per
seq ← list()
function GenerateSequenceList(seq, row_{op}, col_{op})
    return seq
layers ← list()
function GenerateLayersList(layers, Ls_r, Ls_c)
    return layers
correct ← Verify($origin$, layers, seq, mat)
if !correct **then**
    return
return d, layers

List of Figures

List of Tables

Temporary page!

LaTeX was unable to guess the total number of pages correctly. As there was some unprocessed data that should have been added to the final page this extra page has been added to receive it.
If you rerun the document (without altering it) this surplus page will go away, because LaTeX now knows how many pages to expect for this document.